UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Segurança Computacional
Prof.: Paulo Henrique Lopes Silva

Prática 6.4 – CRC, SHA e HMAC

1. Caracterização.

- Tanto ***checksums*** quanto **funções de hash criptográficas** produzem um resumo compacto de dados, que é um número ou sequência que representa o conteúdo original.
- *Checksums* (ou funções de verificação de erro) são muito rápidas, mas não seguras.
    - São projetadas apenas para detectar erros acidentais, como:
        - Bits corrompidos em transmissão;
        - Defeitos de leitura em disco;
        - Problemas de rede.

    - Exemplos:
        - CRC32 *(Cyclic Redundancy Check)*: usado em pacotes de rede, arquivos ZIP e sistemas de arquivos.
        - Adler-32: usado no zlib para detectar corrupção de dados.
        - Soma de verificação simples: soma dos bytes de um arquivo.

    - Características:
        - Baseadas em operações aritméticas ou polinomiais simples.
        - Determinísticas, mas previsíveis.
        - Um invasor pode alterar dados e recalcular facilmente o *checksum*, sem ser detectado.
        - Produzem resumos curtos (32 bits, 64 bits).

- Hashes criptográficos: Funções de hash criptográficas são projetadas com requisitos de segurança rigorosos, como:
    - Unidirecionalidade: não é possível obter a mensagem original a partir do hash.
    - Resistência a colisões: É difícil encontrar duas mensagens diferentes com o mesmo hash.
    - Efeito avalanche: pequenas mudanças no dado geram grandes mudanças no hash.
    - Impossibilidade de manipulação intencional: mesmo conhecendo o algoritmo, não é possível forjar uma mensagem com o mesmo hash.

    - Exemplos:
        - SHA-2 (SHA-256, SHA-512): amplamente usado hoje (assinaturas, certificados, blockchain).
        - SHA-3 (Keccak): nova geração com estrutura interna diferente.
        - MD5 e SHA-1: já quebrados, não devem ser usados em segurança, mas servem para fins de compatibilidade e aprendizado.

    - Características:
        - Produzem resumos longos (128 a 512 bits).
        - Projetadas para resistir a ataques deliberados.
        - Usadas em:
            - Armazenamento de senhas (com *salt*);
            - Verificação de integridade de arquivos;
            - Assinaturas digitais e certificados;
            - Blockchain e criptomoedas.

2. Calcular CRC32, SHA-256 e HMAC-SHA256 de mensagens.

Java

```java
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.security.MessageDigest;
import java.util.zip.CRC32;

public class Util {
    // Converte array de bytes para string hex (minúscula),
    // garantindo que cada byte aparece com dois dígitos (00..ff).
    public static String bytes2Hex(byte[] bytes) {
        StringBuilder sb = new StringBuilder(bytes.length * 2);
        // aloca tamanho aproximado
        for (byte b : bytes) {
            sb.append(String.format("%02x", b & 0xFF));
            // formata como dois hex dígitos
        }
        return sb.toString();
    }

    // Calcula CRC32 de um array de bytes e retorna o valor (32 bits) como long
    public static long calcularCRC32(byte[] bytesMensagem) {
        CRC32 crc = new CRC32();
        // cria um objeto CRC32
        crc.update(bytesMensagem);
        // atualiza com os bytes da mensagem
        return crc.getValue();
        // obtém o valor (0 .. 2^32-1) como long
    }

    // Calcula SHA-256 (digest completo) e retorna o array de bytes do digest
    public static byte[] calcularSHA256(byte[] data) throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        // instancia SHA-256
        return md.digest(data);
        // digest retorna 32 bytes
    }

    // Calcula HMAC-SHA256 dado uma chave (bytes) e os dados (bytes)
    public static byte[] calcularHmacSha256(byte[] chave,
                                            byte[] bytesMensagem) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        // cria instância Mac
        SecretKeySpec keySpec = new SecretKeySpec(chave, "HmacSHA256");
        // cria especificação da chave
        mac.init(keySpec);
        // inicializa o Mac com a chave
        return mac.doFinal(bytesMensagem);
        // processa os dados e retorna o HMAC
    }

    //Verifica HMAC de forma segura (tempo constante usando MessageDigest.isEqual)
     public static boolean checarHmac(byte[] chave, byte[] bytesMensagem, byte[] hmacRecebido)
throws Exception {
        byte[] hmacCalculado = calcularHmacSha256(chave, bytesMensagem);
        // calcula o HMAC real

        return MessageDigest.isEqual(hmacCalculado, hmacRecebido);
        // MessageDigest.isEqual faz comparação segura
    }
}
```

Java

```java
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class Demo1 {
    public static void main(String[] args) throws Exception{

        // Mensagem original (exemplo)
        String mensagemOriginal =
                    "Mensagem de exemplo para demonstrar CRC, SHA-256 e HMAC.";
        // Converte a string para bytes em UTF-8
         byte[] bytesOriginais = mensagemOriginal.getBytes(StandardCharsets.UTF_8);

        // 1. Calcula CRC32
        long valorCRC = Util.calcularCRC32(bytesOriginais);
        // CRC32 como long (0..2^32-1)

        // 2. Calcula SHA-256
        byte[] bytesSHA256 = Util.calcularSHA256(bytesOriginais);
        // 32 bytes do digest SHA-256

        // 3. Calcula HMAC-SHA256 com uma chave secreta (simétrica)
        String chaveSecreta = "chave-secreta-super-segura";
        // exemplo de chave (em app real: não hardcode!)
        byte[] bytesChave = chaveSecreta.getBytes(StandardCharsets.UTF_8);
        // chave em bytes

        // calcula o HMAC da mensagem original
        byte[] bytesHmac = Util.calcularHmacSha256(  , bytesOriginais);

        // --- Impressão dos resultados (forma legível)
        System.out.println("Mensagem original: " + mensagemOriginal);
        System.out.println("Tamanho: " + mensagemOriginal.getBytes().length);
        System.out.println();

        // imprime CRC em hex com 8 dígitos
        // (mas usar & 0xFFFFFFFFL para evitar sinal)
        System.out.printf("CRC32 (hex)    : %08x%n", valorCRC & 0xFFFFFFFFL);
        System.out.println("Tamanho: " +
                    String.valueOf(valorCRC).length());

        // imprime SHA-256 em hex (32 bytes -> 64 hex chars)
        System.out.println("SHA-256 (hex) : " + Util.bytes2Hex(bytesSHA256));
        System.out.println("Tamanho: " + bytesSHA256.length);

        // imprime HMAC em hex e em Base64 (Base64 é útil para transmissões/texto)
        System.out.println("HMAC-SHA256 (hex) : " + Util.bytes2Hex(bytesHmac));
        System.out.println("Tamanho: " + bytesHmac.length);
        System.out.println("HMAC-SHA256 (b64) : " +
                Base64.getEncoder().encodeToString(bytesHmac));
        System.out.println();

        // Simulação 1: alteração simples da mensagem (um atacante muda o texto)

         String mensagemAlterada =
                    "Mensagem de exemplo para demonstrar CRC, SHA-256 e HMAC?";
        // note '?' no final
        byte[] bytesMsgAlterada =
                    mensagemAlterada.getBytes(StandardCharsets.UTF_8);

        // atacante pode calcular CRC e SHA-256 facilmente (sem chave)
        long crcMsgAlterada = Util.calcularCRC32(bytesMsgAlterada);
        byte[] shaMsgAlterada = Util.calcularSHA256(bytesMsgAlterada);

        // atacante NÃO pode calcular HMAC válido a menos que possua a chave
        // aqui o HMAC é obtido com a chave correta
```

```
        byte[] hmacMsgAlteradaComChave =
                    Util.calcularHmacSha256(bytesChave, bytesMsgAlterada);

        System.out.println("Mensagem alterada pelo atacante: " +
                    mensagemAlterada);
        System.out.printf("CRC32 (hex)   : %08x%n", crcMsgAlterada & 0xFFFFFFFFL);
        System.out.println("SHA-256 (hex) : " +
                Util.bytes2Hex(shaMsgAlterada));
        System.out.println("HMAC-SHA256 (hex) [se atacante conhecer a chave] : "
                + Util.bytes2Hex(hmacMsgAlteradaComChave));
        System.out.println();

    }
}
```

Java

```
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Base64;

public class Demo2 {
    public static void main(String[] args) throws Exception{

        // Mensagem original (exemplo)
        String mensagemOriginal =
                "Mensagem de exemplo para demonstrar CRC, SHA-256 e HMAC.";
        // Converte a string para bytes em UTF-8
        byte[] bytesOriginais = mensagemOriginal.getBytes(StandardCharsets.UTF_8);

        // 1. Calcula CRC32
        long valorCRC = Util.calcularCRC32(bytesOriginais);
        // CRC32 como long (0..2^32-1)

        // 2. Calcula SHA-256
        byte[] bytesSHA256 = Util.calcularSHA256(bytesOriginais);
        // 32 bytes do digest SHA-256

        // 3. Calcula HMAC-SHA256 com uma chave secreta (simétrica)
        String chaveSecreta = "chave-secreta-super-segura";
        // exemplo de chave (em app real: não hardcode!)
        byte[] bytesChave = chaveSecreta.getBytes(StandardCharsets.UTF_8);
        // chave em bytes

        // calcula o HMAC da mensagem original
        byte[] bytesHmac = Util.calcularHmacSha256(bytesChave, bytesOriginais);

        // --- Impressão dos resultados (forma legível)
        System.out.println("Mensagem original: " + mensagemOriginal);
        System.out.println();

        System.out.printf("CRC32 (hex)   : %08x%n", valorCRC & 0xFFFFFFFFL);

        System.out.println("SHA-256 (hex) : " + Util.bytes2Hex(bytesSHA256));

        System.out.println("HMAC-SHA256 (hex) : " +
                Util.bytes2Hex(bytesHmac));
        System.out.println("HMAC-SHA256 (b64) : " +
                Base64.getEncoder().encodeToString(bytesHmac));
        System.out.println();

        // Simulação 2: verificação - receptor calcula e compara
```

```
        // Receptor tem: mensagem recebida, o CRC ou SHA armazenado
         // e o HMAC (se houver)
        // Aqui o receptor verifica integridade/autenticidade:

        // 1. Verificar CRC simples (apenas integridade acidental)
        boolean comparaCRC =
                ((Util.calcularCRC32(bytesOriginais) & 0xFFFFFFFFL)
                        == (valorCRC & 0xFFFFFFFFL) );
        System.out.println("Verificação (CRC) da mensagem original: " +
                (comparaCRC ? "OK" : "FALHOU"));

        // 2. Verificar SHA-256 (integridade) - comparar bytes
        boolean comparaSHA =
                Arrays.equals(Util.calcularSHA256(bytesOriginais), bytesSHA256);
        System.out.println("Verificação (SHA-256) da mensagem original: " +
                (comparaSHA ? "OK" : "FALHOU"));

        // 3. Verificar HMAC: receptor precisa da chave secreta
        // para recalcular e comparar
        boolean hmacValido =
               Util.checarHmac(bytesChave, bytesOriginais, bytesHmac);
        System.out.println("Verificação (HMAC) da mensagem original com a chave
               correta: " + (hmacValido ? "OK" : "FALHOU"));

        // 4. Se o receptor usa chave errada
        // (simula atacante que tenta forjar HMAC com outra chave)
        byte[] chaveIntruso =
               "chave-do-atacante".getBytes(StandardCharsets.UTF_8);
        boolean hmacComChaveErrada =
                Util.checarHmac(chaveIntruso, bytesOriginais, bytesHmac);
        System.out.println("Verificação (HMAC) com chave errada (simulação de
               ataque): " + (hmacComChaveErrada ?
               "OK (inseguro!)" : "INVÁLIDO - correção esperada"));

    }
}
```

3. Desenvolver uma aplicação cliente/servidor.

- Fluxo 1:
  - O cliente envia uma mensagem (texto).
  - O servidor calcula e devolve:
    - o CRC32.
    - o SHA-256.
    - o HMAC-SHA256 (usando uma chave secreta compartilhada).

- Fluxo 2:
  - Um cliente “bom” envia mensagens autênticas com a chave privada correta (para gerar o hmac).
  - Um cliente “ruim” envia mensagens com uma chave diferente da real (pois ele não conhece a real) para tentar enviar mensagens autênticas.
  - O servidor recebe as mensagens e faz a verificação do hmac (visto em sala).
    - Ele deve descartar mensagens com hmac inválido.

Referências.

STALLINGS, William. Cryptography and Network Security: Principles and Practice. 8. ed. Pearson, 2023.

ORACLE. Java Platform, Standard Edition 17 API Specification. Oracle Corporation, 2024. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/

RFC 2104 – KRAWCZYK, H.; BELLARE, M.; CANETTI, R. HMAC: Keyed-Hashing for Message Authentication. Internet Engineering Task Force (IETF), 1997. Disponível em: https://www.rfc-editor.org/rfc/rfc2104.